

EDA DATA CLEANING MASTER GUIDE

*A Complete, Practical Handbook for Exploratory Data Analysis
and Data Cleaning with Python — Pandas & NumPy*

Real Datasets • Real Code • Real Interview Prep

Housing • Banking • Healthcare • Retail • HR • Restaurants

Table of Contents

Right-click and choose "Update Field" (or press F9) after opening in Word to refresh page numbers.

Table of Contents	2
Chapter 1: Introduction to Data Cleaning	7
1.1 What is Data Cleaning?	7
1.2 Why Data Cleaning Matters	7
1.3 The EDA + Data Cleaning Relationship	7
1.4 Common Types of Data Problems	8
1.5 Tools We Will Use.....	8
Chapter 2: Loading & Exploring Data	9
2.1 Reading Different File Formats	9
Key Parameters You Will Actually Use	9
2.2 Saving Cleaned Data	10
2.3 First Look at a Dataset.....	10
Chapter 3: Understanding Your Dataset	11
3.1 df.info() — Your Dataset's ID Card	11
3.2 df.describe() — Summary Statistics	11
3.3 Other Essential Exploration Commands.....	11
Chapter 4: Handling Missing Values.....	13
4.1 Why Data Goes Missing.....	13
4.2 Detecting Missing Values	13
4.3 Strategy #1 — Deleting Missing Data.....	14
dropna() — Removing Rows or Columns	14
4.4 Strategy #2 — Imputing (Filling) Missing Data.....	14
fillna() — Basic Filling	14
Filling with Statistics (Mean, Median, Mode)	14
Group-wise Imputation (More Accurate).....	15
Interpolation — For Ordered/Time-Series Data	15
4.5 Strategy #3 — Flagging Missingness	15
4.6 Advanced: Predictive Imputation.....	15
4.7 Decision Framework: Which Method Should You Use?.....	16
Chapter 5: Removing Duplicate Data	17
5.1 Why Duplicates Happen	17
5.2 Detecting Duplicates	17
keep Parameter Explained	17

5.3 Removing Duplicates.....	17
5.4 Fuzzy / Near-Duplicates.....	17
5.5 Real-World Example: Sales Data	18
Chapter 6: Fixing Data Types.....	19
6.1 Why Data Types Matter	19
6.2 Checking Types.....	19
6.3 Converting Types with <code>astype()</code>	19
6.4 Cleaning Numbers Stored as Text	19
6.5 <code>pd.to_numeric()</code> — The Safer Converter.....	19
6.6 The category Type — When and Why.....	20
6.7 Boolean Conversion	20
Chapter 7: Cleaning Text Data.....	21
7.1 Why Text Data Gets Messy	21
7.2 The <code>.str</code> Accessor — Core Functions.....	21
7.3 Splitting and Combining Columns	21
7.4 Regex Cleaning — Removing Unwanted Characters.....	21
7.5 Fixing Encoding Issues	22
Chapter 8: Handling Outliers.....	23
8.1 What is an Outlier?.....	23
8.2 Detecting Outliers Visually	23
8.3 Detecting Outliers Statistically — IQR Method	23
8.4 Detecting Outliers Statistically — Z-Score Method.....	23
8.5 Handling Outliers: Four Options.....	24
Option 1: Remove Them	24
Option 2: Cap Them (Winsorization).....	24
Option 3: Transform the Data	24
Option 4: Keep Them (Investigate Instead)	24
Chapter 9: Cleaning Categorical Data	25
9.1 The Problem: Inconsistent Categories	25
9.2 Standardizing Text Categories.....	25
9.3 <code>replace()</code> vs <code>map()</code>	25
9.4 Grouping Rare Categories	25
9.5 One-Hot and Label Encoding (Prep for Modeling)	26
Chapter 10: Column Operations	27
10.1 Renaming Columns.....	27
10.2 Adding, Dropping, and Reordering Columns.....	27
10.3 <code>apply()</code> , <code>map()</code> , and <code>applymap()</code>	27

10.4 Creating Bins with <code>pd.cut()</code> and <code>pd.qcut()</code>	28
Chapter 11: Filtering Data	29
11.1 Boolean Indexing Basics	29
11.2 <code>isin()</code> and <code>between()</code>	29
11.3 <code>query()</code> — Readable Filtering	29
11.4 <code>loc[]</code> vs <code>iloc[]</code>	29
Chapter 12: Cleaning Numeric Values.....	31
12.1 Validating Numeric Ranges	31
12.2 Fixing Negative Values That Should Be Positive.....	31
12.3 Rounding and Precision.....	31
12.4 Scaling and Normalization (Prep for Modeling)	31
12.5 Handling Infinite Values	32
Chapter 13: Cleaning Dates	33
13.1 Converting Text to Datetime	33
13.2 Extracting Date Parts.....	33
13.3 Validating Date Logic.....	33
13.4 Handling Time Zones.....	33
13.5 Working with Date Ranges	33
Chapter 14: Complete End-to-End Data Cleaning Example	35
14.1 The Raw Dataset.....	35
14.2 Step-by-Step Cleaning	35
Chapter 15: Pandas Data Cleaning Cheat Sheet	37
15.1 Loading & Inspection.....	37
15.2 Missing Values.....	37
15.3 Duplicates	37
15.4 Data Types.....	38
15.5 Text Cleaning (<code>.str</code> accessor).....	38
15.6 Outliers & Numeric Cleaning.....	38
15.7 Categorical Cleaning.....	39
15.8 Columns & Filtering	39
15.9 Dates	39
15.10 Export	40
Chapter 16: Common Beginner Mistakes	41
1. Chained indexing assignment.....	41
2. Forgetting inplace vs reassignment.....	41
3. Not checking dtypes after loading	41
4. Filling missing values before understanding why they're missing	41

5. Dropping rows with dropna() with no subset	41
6. Using mean() to fill skewed data	41
7. Ignoring leading/trailing spaces in text	41
8. Comparing dates/strings without converting types.....	41
9. Not resetting the index after filtering	41
10. Deleting outliers without investigation.....	41
11. Applying one-hot encoding to high-cardinality columns	41
12. Not saving intermediate cleaned versions.....	42
Chapter 17: Data Cleaning Interview Questions	43
17.1 Beginner Level	43
Q1. What is data cleaning and why is it important?	43
Q2. How do you check for missing values in a DataFrame?	43
Q3. What is the difference between dropna() and fillna()?.....	43
Q4. How do you find and remove duplicate rows?.....	43
Q5. What does df.info() show you?	43
17.2 Intermediate Level	43
Q1. Explain MCAR, MAR, and MNAR.	43
Q2. When would you use median instead of mean to fill missing values?.....	43
Q3. How do you detect outliers, and what do you do with them?.....	43
Q4. What is the difference between .loc[] and .iloc[]?	43
Q5. Why is chained indexing (df[df.x>5]["y"]=1) discouraged?	43
Q6. How would you standardize inconsistent category values like "NY", "New York", "new york"?	43
17.3 Advanced Level.....	44
Q1. How would you design a cleaning pipeline that is repeatable across many similar datasets?	44
Q2. How do you decide between deleting, imputing, or flagging missing data at scale?	44
Q3. How would you validate that a cleaning pipeline hasn't introduced new bugs?.....	44
Q4. A dataset has millions of rows and .apply() is too slow — what do you do?	44
Q5. How do you handle schema drift when the same data source changes column names or types over time?	44
Chapter 18: Complete Data Cleaning Roadmap for Any Dataset	45
18.1 Detailed Notes for Every Step	46
Load Dataset.....	46
Understand Dataset	46
Inspect Shape & Columns.....	46
Check Data Types	46
Analyze Missing Values	46
Handle Missing Values	47

Check Duplicate Records	47
Fix Data Types	47
Clean Text Columns	47
Standardize Categories.....	47
Handle Outliers.....	47
Validate Numeric Ranges	48
Clean Date Columns	48
Rename & Organize Columns.....	48
Final Data Quality Checks	48
Export Clean Dataset	48
Chapter 19: Practice Datasets and Exercises	49
Housing Prices Dataset.....	49
Customer Information Dataset	49
Banking Transactions Dataset	49
Employee Records Dataset	49
Medical Records Dataset.....	49
Product Inventory Dataset	50
Restaurant Reviews Dataset	50
Chapter 20: Final 10-15 Minute Revision Sheet.....	51
The Roadmap (Memorize This Order).....	51
Missing Values — 3 Strategies	51
Outliers — 4 Options.....	51
Key One-Liners to Remember	51
Interview Golden Rules	51
You're Ready	52

Chapter 1: Introduction to Data Cleaning

1.1 What is Data Cleaning?

Data cleaning is the process of finding and fixing (or removing) incorrect, incomplete, duplicated, or badly formatted data inside a dataset. Think of raw data like vegetables straight from a farm — before you cook with them, you wash off the mud, cut off the bad parts, and sort them. Data cleaning is that "washing" step for data.

In real companies, data almost never arrives clean. It comes from web forms filled by careless users, sensors that occasionally glitch, systems that merge over the years, and humans who make typos. Before any analysis, dashboard, or machine learning model can be trusted, the data behind it must be cleaned.

1.2 Why Data Cleaning Matters

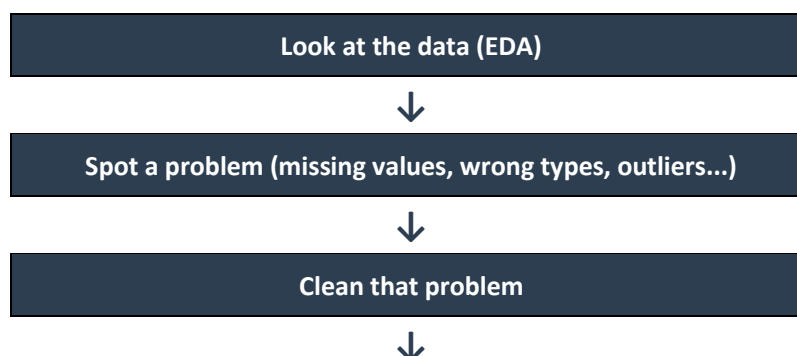
- "Garbage in, garbage out" — a beautiful model trained on dirty data will produce wrong, misleading predictions.
- Studies commonly cite that data scientists spend 60-80% of their time cleaning and preparing data, not building models.
- Bad data leads to bad business decisions — e.g., a retail chain restocking the wrong products because sales figures had duplicate rows.
- Clean data builds trust — stakeholders stop trusting a report the moment they spot an obvious error.

INTERVIEW TIP

A very common interview opener is: "Why is data cleaning important?" Answer with the phrase "garbage in, garbage out" and mention that most real-world time is spent here, not on modeling.

1.3 The EDA + Data Cleaning Relationship

Exploratory Data Analysis (EDA) and Data Cleaning are best friends who work together in a loop, not a straight line:



You repeat this loop — explore, spot an issue, fix it, explore again — until the dataset is trustworthy. This guide focuses heavily on the "fix it" side, since that is where most beginners get stuck.

1.4 Common Types of Data Problems

Problem Type	Example	Typical Dataset
Missing values	Customer age left blank	Customer Info
Duplicate rows	Same order recorded twice	Sales Data
Wrong data type	Price stored as text "₹1,200"	Product Inventory
Inconsistent text	"New York", "new york", "NYC"	Housing Prices
Outliers	Salary of ₹5,00,00,000 for an intern	Employee Records
Invalid dates	Birthdate in the year 2099	Medical Records
Inconsistent categories	"M", "Male", "male", "MALE"	Banking Data
Structural errors	Extra spaces, mismatched columns	Restaurant Reviews

1.5 Tools We Will Use

This guide uses Python with two core libraries:

- Pandas — the primary tool for loading, exploring, and cleaning tabular data (rows and columns).
- NumPy — used underneath Pandas for fast numeric operations, and directly for things like NaN handling and array math.

```
import pandas as pd
import numpy as np

print(pd.__version__)
print(np.__version__)
```

PRO TIP

Always print `pd.__version__` when starting a new project. Some functions (like `df.convert_dtypes()`) behave differently across Pandas versions.

Chapter 2: Loading & Exploring Data

2.1 Reading Different File Formats

Before cleaning anything, you need to get the data into a Pandas DataFrame. Pandas can read almost any format you will meet in industry.

```
# CSV - the most common format
sales = pd.read_csv("sales_data.csv")

# Excel - useful for finance/banking data
bank = pd.read_excel("banking_data.xlsx", sheet_name="Transactions")

# JSON - common for API/web data
reviews = pd.read_json("restaurant_reviews.json")

# SQL database
import sqlite3
conn = sqlite3.connect("employees.db")
employees = pd.read_sql("SELECT * FROM employee_records", conn)
```

Key Parameters You Will Actually Use

Parameter	Function	Why It Matters
sep / delimiter	read_csv	Some files use ";" or tab instead of ","
header	read_csv, read_excel	Tells Pandas which row has column names
na_values	read_csv	Treats custom strings like "N/A", "--" as missing
dtype	read_csv	Forces a column to load as a specific type (e.g., str for zip codes)
parse_dates	read_csv	Automatically converts date-like columns to datetime
usecols	read_csv	Loads only the columns you need — saves memory
nrows	read_csv	Loads only a sample of rows for quick testing

```
# Real-world housing dataset load with smart options
housing = pd.read_csv(
    "housing_prices.csv",
    na_values=["N/A", "--", "unknown", ""],
    dtype={"zip_code": str},
    parse_dates=["listed_date"]
)
```

 WARNING

Zip codes, phone numbers, and IDs should almost always be loaded as strings (str), not integers. Otherwise "007" becomes 7, and leading zeros are silently lost forever.

2.2 Saving Cleaned Data

```
sales.to_csv("sales_data_clean.csv", index=False)
bank.to_excel("banking_data_clean.xlsx", index=False)
```

 COMMON MISTAKE

Forgetting `index=False` when saving adds an extra unnamed column ("Unnamed: 0") every time the file is reloaded. This is one of the most common beginner mistakes.

2.3 First Look at a Dataset

The moment a dataset loads, run this "starter kit" of five commands. This applies to any dataset — housing, banking, employees, anything.

```
df.head()      # first 5 rows - sanity check
df.tail()      # last 5 rows - check for footer junk/totals
df.shape       # (rows, columns)
df.columns     # list of column names
df.sample(5)   # 5 RANDOM rows - often reveals problems head() hides
```

 PRO TIP

`df.head()` only shows the top rows, which are often clean because they were entered first/carefully. Always also run `df.sample(10)` to catch messy rows hiding in the middle of a large file.

Chapter 3: Understanding Your Dataset

3.1 df.info() — Your Dataset's ID Card

`df.info()` is usually the very first command run after loading. It tells you the row count, column names, non-null counts, and data types all at once — like a dataset's ID card.

```
employees = pd.read_csv("employee_records.csv")
employees.info()
```

```
# <class 'pandas.core.frame.DataFrame'>
# RangeIndex: 1000 entries, 0 to 999
# Data columns (total 6 columns):
# #   Column      Non-Null Count  Dtype
# ---  -
# 0   emp_id      1000 non-null   int64
# 1   name        998 non-null    object
# 2   department  1000 non-null    object
# 3   salary      985 non-null    float64
# 4   join_date   1000 non-null    object
# 5   rating      910 non-null    float64
```

From this single command you instantly know: 2 missing names, 15 missing salaries, 90 missing ratings, and `join_date` is loaded as text (object) instead of a proper date — a red flag to fix in Chapter 13.

3.2 df.describe() — Summary Statistics

```
employees["salary"].describe()
# count      985.000000
# mean      58234.120000
# std       21453.870000
# min       -500.000000  <- negative salary?! impossible, an error
# 25%       42000.000000
# 50%       56000.000000
# 75%       71000.000000
# max      999999.000000  <- extreme outlier, likely a data entry error
```

`describe()` instantly reveals two red flags: a negative salary (min) and an absurdly high salary (max). This is your first clue that outlier handling (Chapter 8) will be needed.

- Use `describe(include="object")` to get summary stats for text/categorical columns (count, unique, top, freq).
- Use `describe(include="all")` to see everything at once.

3.3 Other Essential Exploration Commands

Command	What It Tells You
<code>df.dtypes</code>	Data type of every column
<code>df.isnull().sum()</code>	Count of missing values per column

<code>df.nunique()</code>	Number of unique values per column
<code>df["col"].value_counts()</code>	Frequency of each category
<code>df.duplicated().sum()</code>	Number of fully duplicated rows
<code>df.memory_usage(deep=True)</code>	Memory used per column (helps with big data)
<code>df.corr(numeric_only=True)</code>	Correlation between numeric columns

```
# Restaurant review dataset example
reviews["rating"].value_counts()
# 5.0    412
# 4.0    301
# 3.0    120
# 1.0     45
# 2.0     40
# 0.0      3  <- suspicious: ratings are usually 1-5, not 0

reviews["city"].nunique()  # how many unique cities are reviewed
```

INTERVIEW TIP

A favorite interview question: "How do you get a quick overview of a new dataset?" A strong answer lists: `shape`, `info()`, `describe()`, `isnull().sum()`, `duplicated().sum()`, and `nunique()` — in that order.

Chapter 4: Handling Missing Values

This is the single most important chapter in this guide. Nearly every real dataset has missing values, and how you handle them directly affects the accuracy of any analysis or model built afterward.

4.1 Why Data Goes Missing

Type	Meaning	Example
MCAR (Missing Completely At Random)	No pattern at all; pure chance	A sensor randomly drops one reading
MAR (Missing At Random)	Missingness relates to another observed column	Income missing more often for younger customers
MNAR (Missing Not At Random)	Missingness relates to the value itself	People with very high income refuse to disclose it

INTERVIEW TIP

MCAR vs MAR vs MNAR is a classic intermediate interview question. Explain it with the income example above — it is easy to remember and shows real understanding, not memorization.

4.2 Detecting Missing Values

```
customers = pd.read_csv("customer_information.csv")

customers.isnull().sum()           # missing count per column
customers.isnull().mean() * 100    # missing PERCENTAGE per column
customers.isnull().sum().sort_values(ascending=False)

# Visual pattern (great for reports)
import matplotlib.pyplot as plt
customers.isnull().sum().plot(kind="bar")
plt.title("Missing Values per Column")
plt.show()
```

WARNING

Missing values are not always NaN! Real datasets often disguise missing data as "N/A", "-", "unknown", "NULL", "?", 999, or even empty strings "". Always check `unique()` values on suspicious columns, not just `isnull()`.

```
# Catching "disguised" missing values in a banking dataset
bank["account_type"].unique()
# array(['Savings', 'Current', 'unknown', '?', 'Savings ', nan], dtype=object)

# Convert disguised missing values into real NaN
bank["account_type"] = bank["account_type"].replace(["unknown", "?", ""], np.nan)
```

4.3 Strategy #1 — Deleting Missing Data

dropna() — Removing Rows or Columns

```
df.dropna()           # drop ANY row with at least one NaN
df.dropna(subset=["salary"]) # drop rows only if 'salary' is NaN
df.dropna(axis=1)      # drop COLUMNS that contain NaN
df.dropna(thresh=4)    # keep rows with at least 4 non-null values
df.dropna(how="all")   # drop rows where EVERY value is NaN
```

When to Use dropna()	When NOT to Use dropna()
Missing data is a tiny fraction (<5%) of rows	Missing data is a large chunk (>30%) of rows
The column is not critical to the analysis	The missing pattern is NOT random (MNAR) — deleting biases the data
You have plenty of data to spare	Dataset is already small (deleting shrinks it further)

✗ COMMON MISTAKE

Beginners often run `df.dropna()` with no arguments on the whole DataFrame. If even one column is messy with lots of NaNs, this can silently delete most of your dataset.

4.4 Strategy #2 — Imputing (Filling) Missing Data

fillna() — Basic Filling

```
# Housing dataset: fill missing 'garage_size' with 0 (no garage)
housing["garage_size"] = housing["garage_size"].fillna(0)

# Fill missing text category with a label
customers["membership_tier"] = customers["membership_tier"].fillna("Unknown")

# Forward-fill / backward-fill (great for time series, e.g., stock prices)
sales["daily_price"] = sales["daily_price"].fillna(method="ffill")
sales["daily_price"] = sales["daily_price"].fillna(method="bfill")
```

Filling with Statistics (Mean, Median, Mode)

```
# Mean - good for normally distributed numeric data
employees["salary"] = employees["salary"].fillna(employees["salary"].mean())

# Median - BETTER when outliers exist (median resists extreme values)
employees["salary"] = employees["salary"].fillna(employees["salary"].median())

# Mode - for categorical columns
customers["city"] = customers["city"].fillna(customers["city"].mode()[0])
```

Method	Best For	Watch Out For
--------	----------	---------------

Mean	Symmetric numeric data (e.g., height)	Distorted heavily by outliers (e.g., salary)
Median	Skewed numeric data (e.g., salary, house price)	Ignores the true distribution shape
Mode	Categorical data (e.g., city, gender)	Can over-represent the most common category
Forward/Backward Fill	Time series data (e.g., daily stock price)	Wrong for cross-sectional (non-time) data

Group-wise Imputation (More Accurate)

Filling a single global mean often hides real patterns. A smarter approach fills missing values using the mean/median of a related group — for example, filling a missing salary using the median salary of that employee's department, not the whole company.

```
# Medical dataset: fill missing 'bmi' using average BMI of the same age group
medical["bmi"] = medical.groupby("age_group")["bmi"].transform(
    lambda x: x.fillna(x.median())
)

# Employee dataset: fill missing salary by department median
employees["salary"] = employees.groupby("department")["salary"].transform(
    lambda x: x.fillna(x.median())
)
```

PRO TIP

Group-wise imputation using `groupby().transform()` is a favorite technique to mention in interviews — it shows you think beyond the basic "just fill the mean" answer.

Interpolation — For Ordered/Time-Series Data

```
# Sales data: fill missing daily revenue by interpolating between known points
sales["daily_revenue"] = sales["daily_revenue"].interpolate(method="linear")
```

4.5 Strategy #3 — Flagging Missingness

Sometimes the fact that data is missing is itself useful information for a model. Instead of just filling the value, create an extra column that flags it.

```
medical["bmi_was_missing"] = medical["bmi"].isnull().astype(int)
medical["bmi"] = medical["bmi"].fillna(medical["bmi"].median())
```

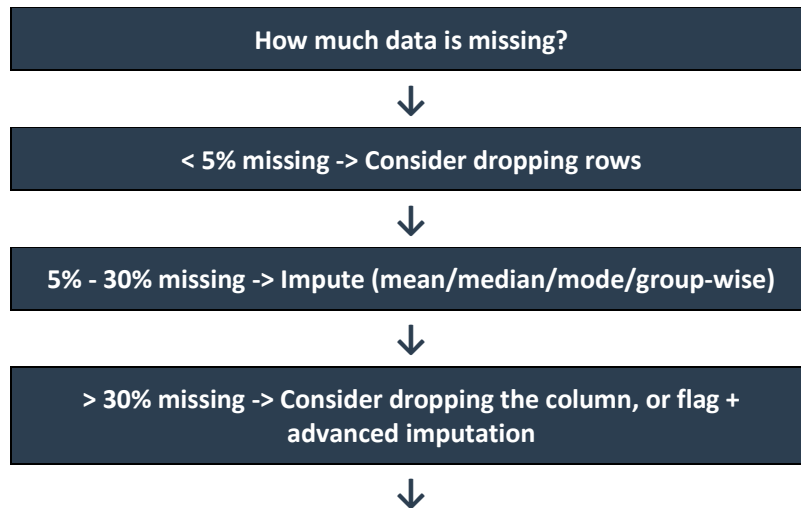
4.6 Advanced: Predictive Imputation

For high-stakes projects, missing values can be predicted using a model trained on the other columns (e.g., scikit-learn's `KNNImputer` or `IterativeImputer`). This is more accurate but heavier — mention it in interviews as the "advanced" answer.

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=5)
housing[["price", "sqft", "bedrooms"]] = imputer.fit_transform(
    housing[["price", "sqft", "bedrooms"]]
)
```

4.7 Decision Framework: Which Method Should You Use?



INTERVIEW TIP

When asked "how do you handle missing data?", never give a one-line answer like "I use fillna()". Walk through the decision framework: detect -> understand WHY it's missing -> choose delete vs impute vs flag based on % missing and importance of the column.

Chapter 5: Removing Duplicate Data

5.1 Why Duplicates Happen

- A user double-clicks "Submit" on a web form (Customer Info dataset)
- Data merged from two different systems that both had the same records (Sales Data)
- A scraping script re-collects the same restaurant review twice (Restaurant Reviews)
- An ETL pipeline re-runs and re-inserts a batch of rows (Banking Data)

5.2 Detecting Duplicates

```
sales = pd.read_csv("sales_data.csv")

sales.duplicated().sum()           # total fully-duplicated rows
sales[sales.duplicated()]         # view the duplicate rows
sales.duplicated(subset=["order_id"]).sum() # duplicates based on ONE key column
sales.duplicated(keep=False)      # marks ALL copies (not just extras) as True
```

keep Parameter Explained

keep value	Behavior
"first" (default)	Marks all duplicates as True EXCEPT the first occurrence
"last"	Marks all duplicates as True EXCEPT the last occurrence
False	Marks ALL occurrences (including the original) as True

5.3 Removing Duplicates

```
sales_clean = sales.drop_duplicates()           # remove exact duplicate rows
sales_clean = sales.drop_duplicates(subset=["order_id"]) # remove based on key column(s)
sales_clean = sales.drop_duplicates(subset=["order_id"], keep="last") # keep the latest entry
sales_clean = sales.drop_duplicates(inplace=False) # inplace=True modifies original df directly
```

WARNING

`drop_duplicates()` with no subset checks EVERY column. Two genuinely different customer records that happen to share the same name and city (but different IDs) will NOT be flagged — always pick the right subset of key columns.

5.4 Fuzzy / Near-Duplicates

Sometimes rows are "logically" duplicates but not textually identical — e.g., "Jon Smith" vs "John Smith", or "123 Main St." vs "123 Main Street". Exact-match `drop_duplicates()` will miss these.

```
# Normalize text first, THEN check for duplicates
customers["name_clean"] = customers["name"].str.lower().str.strip()
customers.duplicated(subset=["name_clean", "email"]).sum()

# For true fuzzy matching, libraries like 'thefuzz' (fuzzywuzzy) or
# recordlinkage are used in production systems
from thefuzz import fuzz
fuzz.ratio("Jon Smith", "John Smith") # returns a similarity score 0-100
```

5.5 Real-World Example: Sales Data

```
sales = pd.read_csv("sales_data.csv")
print("Before:", sales.shape)

# Business rule: an order_id should only ever appear once.
# If it appears twice, keep the most recently updated record.
sales = sales.sort_values("last_updated").drop_duplicates(
    subset=["order_id"], keep="last"
)
print("After:", sales.shape)
```

✗ COMMON MISTAKE

Removing duplicates BEFORE deciding which copy is the "correct" one. Always sort by a reliable column (like `last_updated` or a version number) first, so `drop_duplicates` keeps the right record.

🔗 INTERVIEW TIP

Interviewers love asking: "The dataset has duplicate `order_ids` but the row values differ slightly — how do you decide which to keep?" Good answer: check for a timestamp/version column and keep the most recent; if none exists, flag it for the business/data owner rather than guessing.

Chapter 6: Fixing Data Types

6.1 Why Data Types Matter

Pandas guesses a data type for each column when loading. Its guesses are often wrong — a price column with a "\$" sign loads as text (object), a date loads as text, and a category loads as generic object instead of the more efficient category type. Wrong types silently break math, sorting, filtering, and grouping.

6.2 Checking Types

```
df.dtypes
df.info()
```

6.3 Converting Types with `astype()`

```
# Product inventory dataset
inventory["quantity"] = inventory["quantity"].astype(int)
inventory["price"] = inventory["price"].astype(float)
inventory["sku"] = inventory["sku"].astype(str)
inventory["category"] = inventory["category"].astype("category") # memory efficient
```

WARNING

`astype(int)` will FAIL (raise an error) if the column contains NaN, because NaN has no integer representation. Clean/impute missing values first, or use the nullable "Int64" (capital I) type instead of "int64".

6.4 Cleaning Numbers Stored as Text

This is one of the most common real-world type problems: numeric values are stored as text because they contain currency symbols, commas, or units.

```
# Product inventory: price column looks like "$1,299.00"
inventory["price"] = (
    inventory["price"]
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
    .astype(float)
)

# Housing: area column looks like "1200 sqft"
housing["area_sqft"] = (
    housing["area_sqft"]
    .str.replace(" sqft", "", regex=False)
    .astype(int)
)
```

6.5 `pd.to_numeric()` — The Safer Converter

```
# errors='coerce' turns anything unconvertible into NaN instead of crashing
bank["transaction_amount"] = pd.to_numeric(bank["transaction_amount"], errors="coerce")
```

```
# Find what got coerced to NaN (helps spot bad data)
bad_rows = bank[bank["transaction_amount"].isnull()]
```

errors value	Behavior
"raise" (default)	Stops the program with an error on the first bad value
"coerce"	Converts bad values to NaN and continues — safest for cleaning
"ignore"	Leaves the entire column untouched if any value fails (rarely useful)

6.6 The category Type — When and Why

Convert a column to "category" dtype when it has a small, repeating set of values (like department, gender, membership tier). This saves large amounts of memory and speeds up groupby operations.

```
employees["department"] = employees["department"].astype("category")
employees.memory_usage(deep=True) # compare before/after
```

PRO TIP

Rule of thumb: if a column's unique values are less than ~50% of total rows, category dtype usually saves memory. Check with `df["col"].nunique() / len(df)`.

6.7 Boolean Conversion

```
# Banking: "Y"/"N" text to real booleans
bank["is_fraud"] = bank["is_fraud"].map({"Y": True, "N": False})

# Common alternate encodings
customers["is_active"] = customers["is_active"].replace(
    {"yes": True, "no": False, 1: True, 0: False}
)
```

COMMON MISTAKE

Converting a column with `astype(bool)` directly on strings like "False" gives **WRONG** results — because any non-empty string (even the text "False") is truthy in Python. Always map explicit string-to-boolean pairs instead.

Chapter 7: Cleaning Text Data

7.1 Why Text Data Gets Messy

Free-text and label fields (names, cities, review comments, product names) suffer most from human inconsistency: extra spaces, mixed casing, typos, and stray punctuation. The Pandas `.str` accessor is your main toolkit here.

7.2 The `.str` Accessor — Core Functions

```
reviews["comment"] = reviews["comment"].str.strip()      # remove leading/trailing spaces
reviews["comment"] = reviews["comment"].str.lower()      # normalize case
customers["city"] = customers["city"].str.title()        # "new york" -> "New York"
customers["name"] = customers["name"].str.replace(r"\s+", " ", regex=True) # collapse
multiple spaces
```

Function	Purpose	Example
<code>.str.strip()</code>	Removes leading/trailing whitespace	" Delhi " -> "Delhi"
<code>.str.lower()</code> / <code>.upper()</code>	Normalizes case	"DELHI" -> "delhi"
<code>.str.title()</code>	Capitalizes each word	"new york" -> "New York"
<code>.str.replace()</code>	Find & replace substrings/regex	"Rs.500" -> "500"
<code>.str.contains()</code>	Checks if a substring exists	Filter reviews containing "bad"
<code>.str.split()</code>	Splits a string into a list/columns	"John Smith" -> ["John", "Smith"]
<code>.str.len()</code>	Length of each string	Flag comments that are too short
<code>.str.extract()</code>	Pulls out a regex pattern	Extract area code from a phone number

7.3 Splitting and Combining Columns

```
# Split full name into first/last (Customer dataset)
customers[["first_name", "last_name"]] = customers["name"].str.split(" ", n=1, expand=True)

# Combine address parts into one column (Housing dataset)
housing["full_address"] = (
    housing["street"] + ", " + housing["city"] + ", " + housing["zip_code"]
)
```

7.4 Regex Cleaning — Removing Unwanted Characters

```
# Restaurant reviews: strip emojis/special characters, keep letters, numbers, basic
punctuation
reviews["comment"] = reviews["comment"].str.replace(r"[^a-zA-Z0-9\s.,!?'"]", "", regex=True)
```

```
# Banking: extract only digits from a messy phone number field
bank["phone_clean"] = bank["phone"].str.extract(r"(\d{10})")

# Product SKU: standardize format (remove dashes/spaces, uppercase)
inventory["sku"] = inventory["sku"].str.replace(r"[-\s]", "", regex=True).str.upper()
```

PRO TIP

Test regex patterns on 2-3 sample strings using Python directly before applying them to an entire column — a wrong regex can silently corrupt thousands of rows.

7.5 Fixing Encoding Issues

```
# Common when data comes from web scraping or older systems
reviews["comment"] = reviews["comment"].str.encode("ascii", "ignore").str.decode("ascii")

# Or specify correct encoding at load time
reviews = pd.read_csv("restaurant_reviews.csv", encoding="utf-8")
```

COMMON MISTAKE

Applying `.str.lower()` to a column that still contains NaN values usually works fine (NaN stays NaN), but chaining multiple `.str` operations after a non-string type sneaks in (e.g., numbers mixed into a text column) will throw errors. Always confirm dtype is object/string first.

Chapter 8: Handling Outliers

8.1 What is an Outlier?

An outlier is a data point that is very different from the rest of the data. It could be a genuine rare event (a mansion in a housing dataset) or a data entry error (a typo that turned \$50,000 into \$5,000,000). Telling these two apart is the real skill.

8.2 Detecting Outliers Visually

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.boxplot(x=housing["price"])
plt.show()

sns.histplot(housing["price"], bins=50)
plt.show()
```

8.3 Detecting Outliers Statistically — IQR Method

The Interquartile Range (IQR) method is the most common, robust way to flag outliers without assuming a normal distribution.

```
Q1 = housing["price"].quantile(0.25)
Q3 = housing["price"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = housing[(housing["price"] < lower_bound) | (housing["price"] > upper_bound)]
print(f"Found {len(outliers)} outliers")
```

8.4 Detecting Outliers Statistically — Z-Score Method

Best used when the data is roughly normally distributed (bell-curve shaped). A Z-score tells you how many standard deviations a value is from the mean.

```
from scipy import stats

employees["salary_zscore"] = stats.zscore(employees["salary"].dropna())
outliers = employees[employees["salary_zscore"].abs() > 3] # beyond 3 std deviations
```

Method	Best For	Limitation
IQR (Boxplot)	Skewed data, general purpose (default choice)	Fixed 1.5x multiplier is a rule of thumb, not a law

Z-score	Normally distributed data	Sensitive to outliers itself when calculating mean/std
Domain rules	Known valid ranges (e.g., age 0-120)	Requires business/domain knowledge

8.5 Handling Outliers: Four Options

Option 1: Remove Them

```
housing_clean = housing[(housing["price"] >= lower_bound) & (housing["price"] <= upper_bound)]
```

Option 2: Cap Them (Winsorization)

```
housing["price"] = housing["price"].clip(lower=lower_bound, upper=upper_bound)
```

Option 3: Transform the Data

```
# Log transform reduces the impact of extreme values (common for price/income)
housing["price_log"] = np.log1p(housing["price"])
```

Option 4: Keep Them (Investigate Instead)

If the outlier is a genuine, valid data point (e.g., a luxury mansion, a CEO's salary), removing it can hide real business insight. Always ask "is this a mistake or a fact?" before deleting.

WARNING

Never blindly delete outliers before understanding WHY they exist. In a medical dataset, an "outlier" heart rate could indicate a patient who needs urgent attention — deleting it could hide a critical case rather than a data error.

INTERVIEW TIP

When asked "how do you handle outliers?", structure your answer: 1) Detect (IQR/Z-score/visual), 2) Investigate whether it's an error or a real event, 3) Decide: remove, cap, transform, or keep, based on domain context — never a single default action.

Chapter 9: Cleaning Categorical Data

9.1 The Problem: Inconsistent Categories

```
bank["account_type"].unique()
# array(['Savings', 'savings', 'SAVINGS', 'Current', 'current ', 'Curr.'], dtype=object)
```

To a computer, "Savings" and "savings" are two completely different categories. This silently breaks `groupby()`, `value_counts()`, and any model trained on this column.

9.2 Standardizing Text Categories

```
bank["account_type"] = bank["account_type"].str.strip().str.lower()

mapping = {
    "savings": "Savings",
    "current": "Current",
    "curr.": "Current",
    "curr": "Current",
}
bank["account_type"] = bank["account_type"].replace(mapping)
```

9.3 `replace()` vs `map()`

Function	Behavior	Best For
<code>.replace()</code>	Replaces matched values, keeps everything else unchanged	Fixing a few known bad values
<code>.map()</code>	Replaces ALL values using a dictionary; unmapped values become NaN	Full recoding when you know every category

```
# .map() example - full recoding of gender labels (Medical dataset)
medical["gender"] = medical["gender"].map({"M": "Male", "F": "Female"})
# any value NOT in the dict (e.g. "Other") becomes NaN - be careful!
```

✗ COMMON MISTAKE

Using `.map()` when the dictionary doesn't cover every possible category silently turns unmatched values into NaN. Always check `.unique()` BEFORE mapping, or use `.replace()` which leaves unmatched values untouched.

9.4 Grouping Rare Categories

```
# Product inventory: group categories that appear less than 10 times into "Other"
counts = inventory["category"].value_counts()
rare = counts[counts < 10].index
inventory["category"] = inventory["category"].replace(rare, "Other")
```

9.5 One-Hot and Label Encoding (Prep for Modeling)

```
# One-hot encoding - creates a 0/1 column per category (for ML models)
pd.get_dummies(employees["department"], prefix="dept")

# Label encoding - assigns each category an integer (for ordinal data)
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
employees["department_encoded"] = le.fit_transform(employees["department"])
```

PRO TIP

One-hot encoding is for unordered categories (e.g., city). Label/ordinal encoding is for ordered categories (e.g., "Low", "Medium", "High"). Mixing these up is a common mistake that hurts model performance.

Chapter 10: Column Operations

10.1 Renaming Columns

```
employees.rename(columns={"emp_id": "employee_id", "sal": "salary"}, inplace=True)

# Clean ALL column names at once - a great habit for every new dataset
employees.columns = (
    employees.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
```

PRO TIP

Make cleaning column names your very first step on any new dataset — it prevents bugs later from spaces, mixed casing, or special characters in column names.

10.2 Adding, Dropping, and Reordering Columns

```
# Add a new derived column
sales["profit"] = sales["revenue"] - sales["cost"]

# Drop unnecessary columns
sales = sales.drop(columns=["temp_notes", "internal_flag"])

# Reorder columns
sales = sales[["order_id", "customer_id", "revenue", "cost", "profit"]]
```

10.3 `apply()`, `map()`, and `applymap()`

Method	Works On	Use Case
<code>Series.map()</code>	A single column	Simple value-to-value substitution
<code>Series/DataFrame.apply()</code>	A column OR whole DataFrame	Custom function logic, row-wise or column-wise
<code>DataFrame.applymap()</code>	Every cell in a DataFrame	Applying the same simple function to all cells (legacy; use <code>.map</code> on DataFrame in new Pandas)

```
# apply() with a custom function - Employee bonus tier
def bonus_tier(salary):
    if salary > 80000:
        return "High"
    elif salary > 50000:
        return "Medium"
```

```

    return "Low"

employees["bonus_tier"] = employees["salary"].apply(bonus_tier)

# apply() row-wise (axis=1) - combine multiple columns
housing["price_per_sqft"] = housing.apply(lambda row: row["price"] / row["area_sqft"],
axis=1)

```

WARNING

`apply()` with `axis=1` (row-wise) is convenient but slow on large datasets because it loops in Python. Prefer vectorized operations (`housing["price"] / housing["area_sqft"]`) whenever possible — they run 10-100x faster.

10.4 Creating Bins with `pd.cut()` and `pd.qcut()`

```

# Equal-WIDTH bins - Employee age groups
employees["age_group"] = pd.cut(
    employees["age"], bins=[18, 30, 45, 60, 100],
    labels=["18-30", "31-45", "46-60", "60+"]
)

# Equal-FREQUENCY bins - Housing price quartiles
housing["price_quartile"] = pd.qcut(housing["price"], q=4, labels=["Low", "Mid-Low", "Mid-High", "High"])

```

INTERVIEW TIP

`pd.cut()` vs `pd.qcut()` is a common question. `cut()` creates bins of equal WIDTH (value range); `qcut()` creates bins of equal SIZE (equal count of rows in each bin) — know the difference and give an example of each.

Chapter 11: Filtering Data

11.1 Boolean Indexing Basics

```
# Banking: transactions over 50,000
big_transactions = bank[bank["transaction_amount"] > 50000]

# Multiple conditions - use & (and), | (or), ~ (not) with PARENTHESES around each condition
high_value_fraud = bank[(bank["transaction_amount"] > 50000) & (bank["is_fraud"] == True)]
not_savings = bank[~(bank["account_type"] == "Savings")]
```

✗ COMMON MISTAKE

Using Python's regular "and"/"or"/"not" instead of "&"/"|"/"~" on Pandas conditions raises an error or gives wrong results. Also, ALWAYS wrap each condition in parentheses — operator precedence rules differ from plain Python.

11.2 isin() and between()

```
# Filter rows where city is one of several values
target_cities = customers[customers["city"].isin(["Mumbai", "Delhi", "Bengaluru"])]

# Filter a numeric range cleanly
mid_range_homes = housing[housing["price"].between(200000, 500000)]
```

11.3 query() — Readable Filtering

```
high_earners = employees.query("salary > 80000 and department == 'Engineering'")
```

💡 PRO TIP

query() is often easier to read for complex, multi-condition filters and lets you avoid repeating the DataFrame name for every column.

11.4 loc[] vs iloc[]

Method	Selects By	Example
.loc[]	Labels (column names, index labels)	df.loc[3, "salary"]
.iloc[]	Integer position (like array indexing)	df.iloc[0:5, 1:3]

```
employees.loc[employees["department"] == "HR", "salary"] =
employees.loc[employees["department"] == "HR", "salary"] * 1.1
employees.iloc[0:10, 0:3] # first 10 rows, first 3 columns by position
```

 WARNING

Never chain indexing like `df[df["x"]>5]["y"] = 10` — this triggers Pandas' infamous `SettingWithCopyWarning` and may silently fail to update the original `DataFrame`. Always use `.loc[]` for filtered assignment.

Chapter 12: Cleaning Numeric Values

12.1 Validating Numeric Ranges

Every numeric column in the real world has a valid range. Age cannot be negative or 300. A discount percentage cannot exceed 100. Validating these ranges catches silent data-entry errors that `describe()` alone might not flag clearly.

```
# Medical dataset: age must be between 0 and 120
invalid_age = medical[(medical["age"] < 0) | (medical["age"] > 120)]
medical.loc[(medical["age"] < 0) | (medical["age"] > 120), "age"] = np.nan

# Banking: interest rate should be between 0 and 100 (percent)
bank.loc[bank["interest_rate"] > 100, "interest_rate"] = np.nan
```

12.2 Fixing Negative Values That Should Be Positive

```
# Employee salary should never be negative - likely a sign error
employees["salary"] = employees["salary"].abs()
```

WARNING

Do not blindly apply `.abs()` everywhere. A negative "profit" or "temperature" value can be completely valid! Only use `abs()` on columns that are mathematically impossible to be negative.

12.3 Rounding and Precision

```
housing["price"] = housing["price"].round(2)
inventory["unit_cost"] = inventory["unit_cost"].apply(lambda x: round(x, 2))
```

12.4 Scaling and Normalization (Prep for Modeling)

```
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Min-Max scaling -> range [0, 1]
scaler = MinMaxScaler()
housing["price_scaled"] = scaler.fit_transform(housing[["price"]])

# Standardization -> mean 0, std 1
std_scaler = StandardScaler()
employees["salary_standardized"] = std_scaler.fit_transform(employees[["salary"]])
```

Technique	Result Range	Best For
Min-Max Scaling	[0, 1]	Neural networks, distance-based models (KNN)

Standardization (Z-score)	Mean 0, Std 1	Models assuming normal distribution (linear/logistic regression)
---------------------------	---------------	--

12.5 Handling Infinite Values

```
# Division by zero can create inf values (e.g., price_per_sqft when area = 0)
housing.replace([np.inf, -np.inf], np.nan, inplace=True)
```

✗ COMMON MISTAKE

Forgetting to check for `np.inf` after a division operation. `isnull()` does NOT catch infinite values — you must explicitly check with `np.isinf(df["col"]).any()`.

Chapter 13: Cleaning Dates

13.1 Converting Text to Datetime

```
employees["join_date"] = pd.to_datetime(employees["join_date"])

# Mixed/messy formats - let pandas infer, or specify explicitly
housing["listed_date"] = pd.to_datetime(housing["listed_date"], format="%d-%m-%Y",
errors="coerce")
```

PRO TIP

Always pass `errors="coerce"` when converting dates from messy real-world data. Any unparseable date becomes NaT (Not a Time) instead of crashing your script — then you can inspect and fix those rows separately.

13.2 Extracting Date Parts

```
employees["join_year"] = employees["join_date"].dt.year
employees["join_month"] = employees["join_date"].dt.month
employees["join_weekday"] = employees["join_date"].dt.day_name()
employees["tenure_days"] = (pd.Timestamp.today() - employees["join_date"]).dt.days
```

13.3 Validating Date Logic

```
# Medical dataset: birth date cannot be in the future
future_births = medical[medical["birth_date"] > pd.Timestamp.today()]

# Banking: transaction date cannot be before account_open_date
invalid_txns = bank[bank["transaction_date"] < bank["account_open_date"]]
```

WARNING

A very common real error: dates stored inconsistently as both DD-MM-YYYY and MM-DD-YYYY in the same column (common when merging US and non-US data sources). "03-04-2024" is ambiguous — always verify with the data source before parsing.

13.4 Handling Time Zones

```
bank["transaction_time"] = bank["transaction_time"].dt.tz_localize("UTC")
bank["transaction_time_ist"] = bank["transaction_time"].dt.tz_convert("Asia/Kolkata")
```

13.5 Working with Date Ranges

```
# Filter last 30 days of sales
recent_sales = sales[sales["order_date"] >= pd.Timestamp.today() - pd.Timedelta(days=30)]

# Resample daily sales into monthly totals (time series)
```

```
monthly_sales = sales.set_index("order_date")["revenue"].resample("M").sum()
```

 INTERVIEW TIP

"How do you handle inconsistent date formats?" — Good answer: use `pd.to_datetime()` with `errors="coerce"` to catch unparseable values, inspect the resulting NaT rows, and if formats genuinely differ across rows, parse subsets separately with the correct explicit format before combining.

Chapter 14: Complete End-to-End Data Cleaning Example

Let's bring every technique together on one dataset: a messy Retail Sales dataset. This mirrors what a real cleaning script looks like in a Jupyter notebook or production pipeline.

14.1 The Raw Dataset

```
import pandas as pd
import numpy as np

sales = pd.read_csv("retail_sales_raw.csv")
sales.head()
#   order_id  customer_name  city      order_date  price  quantity  category
# 0   1001      " john doe " "New York" "2023/01/15" "$120.50"    3  "Electronics "
# 1   1001      " john doe " "New York" "2023/01/15" "$120.50"    3  "Electronics "
<- duplicate
# 2   1002      "Jane SMITH" "new york" "15-01-2023"  "$45"      -2  "clothing"
<- negative qty
# 3   1003      NaN         "Chicago" "2023/13/40"  "unknown"    5  "Home"
<- invalid date/price
# 4   1004      "Bob Lee"   "Chicago" "2023/02/10"  "$99999999"  1  "electronics"
<- price outlier
```

14.2 Step-by-Step Cleaning

```
# 1. Standardize column names
sales.columns = sales.columns.str.strip().str.lower().str.replace(" ", "_")

# 2. Remove exact duplicate rows
sales = sales.drop_duplicates()

# 3. Clean text columns
sales["customer_name"] = sales["customer_name"].str.strip().str.title()
sales["city"] = sales["city"].str.strip().str.title()
sales["category"] = sales["category"].str.strip().str.lower()

# 4. Fix price column (remove $ and commas, convert to numeric, handle "unknown")
sales["price"] = (
    sales["price"].astype(str)
    .str.replace("$", "", regex=False)
    .str.replace(",", "", regex=False)
)
sales["price"] = pd.to_numeric(sales["price"], errors="coerce")

# 5. Fix negative quantities (data entry sign errors)
sales["quantity"] = sales["quantity"].abs()

# 6. Parse dates safely, flag invalid ones
sales["order_date"] = pd.to_datetime(sales["order_date"], errors="coerce")

# 7. Handle missing values
sales["customer_name"] = sales["customer_name"].fillna("Unknown Customer")
```

```
sales["price"] = sales.groupby("category")["price"].transform(lambda x: x.fillna(x.median()))

# 8. Handle outliers with IQR capping
Q1, Q3 = sales["price"].quantile([0.25, 0.75])
IQR = Q3 - Q1
upper = Q3 + 1.5 * IQR
sales["price"] = sales["price"].clip(upper=upper)

# 9. Standardize categories
sales["category"] = sales["category"].replace({"electronics ": "electronics"})

# 10. Final quality check
print(sales.isnull().sum())
print(sales.duplicated().sum())
print(sales.describe())

# 11. Export clean data
sales.to_csv("retail_sales_clean.csv", index=False)
```

 PRO TIP

Notice the ORDER of operations: structural fixes (column names, duplicates) first, then type/text fixes, THEN missing values and outliers. Cleaning types before imputing prevents string "unknown" from silently becoming a bad statistic.

Chapter 15: Pandas Data Cleaning Cheat Sheet

A fast-reference table of the most-used functions for data cleaning, grouped by task. Use this the night before an interview.

15.1 Loading & Inspection

Function	Purpose
<code>pd.read_csv()</code>	Load a CSV file
<code>pd.read_excel()</code>	Load an Excel file
<code>pd.read_json()</code>	Load a JSON file
<code>pd.read_sql()</code>	Load from a SQL database
<code>df.head()</code> / <code>df.tail()</code>	Preview first/last rows
<code>df.sample()</code>	Random row preview
<code>df.shape</code>	Row/column count
<code>df.info()</code>	Types + non-null counts
<code>df.describe()</code>	Summary statistics
<code>df.dtypes</code>	Column data types
<code>df.columns</code>	Column names
<code>df.memory_usage()</code>	Memory used per column

15.2 Missing Values

Function	Purpose
<code>df.isnull()</code> / <code>df.isna()</code>	Detect missing values
<code>df.notnull()</code>	Detect non-missing values
<code>df.isnull().sum()</code>	Count missing per column
<code>df.dropna()</code>	Remove missing rows/columns
<code>df.fillna()</code>	Fill missing values
<code>df.interpolate()</code>	Fill using interpolation
<code>df.ffill()</code> / <code>df.bfill()</code>	Forward/backward fill
<code>df.groupby().transform()</code>	Group-wise imputation

15.3 Duplicates

Function	Purpose
----------	---------

<code>df.duplicated()</code>	Detect duplicate rows
<code>df.drop_duplicates()</code>	Remove duplicate rows

15.4 Data Types

Function	Purpose
<code>df.astype()</code>	Convert column type
<code>pd.to_numeric()</code>	Safely convert to number
<code>pd.to_datetime()</code>	Safely convert to date
<code>df.convert_dtypes()</code>	Auto-infer best dtypes
<code>df.select_dtypes()</code>	Select columns by type

15.5 Text Cleaning (.str accessor)

Function	Purpose
<code>.str.strip()</code>	Remove whitespace
<code>.str.lower()</code> / <code>.upper()</code> / <code>.title()</code>	Change case
<code>.str.replace()</code>	Find & replace
<code>.str.contains()</code>	Substring search
<code>.str.startswith()</code> / <code>.endswith()</code>	Prefix/suffix check
<code>.str.split()</code>	Split into parts
<code>.str.cat()</code>	Concatenate strings
<code>.str.len()</code>	String length
<code>.str.extract()</code>	Regex extraction
<code>.str.pad()</code> / <code>.str.zfill()</code>	Pad strings (e.g., zip codes)

15.6 Outliers & Numeric Cleaning

Function	Purpose
<code>df.quantile()</code>	Get percentile value
<code>df.clip()</code>	Cap values to a range
<code>scipy.stats.zscore()</code>	Compute Z-scores
<code>np.log1p()</code>	Log-transform (handles 0)
<code>df.abs()</code>	Absolute value

<code>df.round()</code>	Round decimals
<code>MinMaxScaler</code> / <code>StandardScaler</code>	Scale/normalize numeric columns

15.7 Categorical Cleaning

Function	Purpose
<code>.value_counts()</code>	Frequency of categories
<code>.nunique()</code>	Count unique values
<code>.unique()</code>	List unique values
<code>.replace()</code>	Replace specific values
<code>.map()</code>	Full recode via dictionary
<code>pd.get_dummies()</code>	One-hot encoding
<code>LabelEncoder()</code>	Ordinal/label encoding
<code>.astype("category")</code>	Convert to category dtype

15.8 Columns & Filtering

Function	Purpose
<code>df.rename()</code>	Rename columns
<code>df.drop()</code>	Drop rows/columns
<code>df.apply()</code>	Apply custom function
<code>df.applymap()</code> / <code>df.map()</code>	Apply function to every cell
<code>pd.cut()</code>	Equal-width binning
<code>pd.qcut()</code>	Equal-frequency binning
<code>df.loc[]</code> / <code>df.iloc[]</code>	Label/position-based selection
<code>df.query()</code>	Readable filtering
<code>df[col].isin()</code>	Match against a list of values
<code>df[col].between()</code>	Range filtering
<code>df.sort_values()</code>	Sort rows
<code>df.reset_index()</code>	Reset row index
<code>df.set_index()</code>	Set a column as index

15.9 Dates

Function	Purpose
<code>pd.to_datetime()</code>	Convert to datetime
<code>.dt.year / .month / .day</code>	Extract date parts
<code>.dt.day_name()</code>	Day of week name
<code>.dt.tz_localize() / tz_convert()</code>	Time zone handling
<code>pd.Timedelta()</code>	Time duration arithmetic
<code>df.resample()</code>	Aggregate time series by period

15.10 Export

Function	Purpose
<code>df.to_csv()</code>	Save as CSV
<code>df.to_excel()</code>	Save as Excel
<code>df.to_json()</code>	Save as JSON
<code>df.to_sql()</code>	Save to a database

Chapter 16: Common Beginner Mistakes

1. Chained indexing assignment

- **Problem:** `df[df.x>5]["y"] = 1` doesn't reliably update the DataFrame
- **Fix:** Always use `df.loc[condition, "y"] = 1`

2. Forgetting inplace vs reassignment

- **Problem:** `df.dropna(inplace=True)` AND `df = df.dropna()` look similar but behave differently
- **Fix:** Pick one style consistently; reassignment is usually safer/clearer

3. Not checking dtypes after loading

- **Problem:** Numeric-looking columns silently stay as text (object)
- **Fix:** Always run `df.info()` right after loading

4. Filling missing values before understanding why they're missing

- **Problem:** Hides real business patterns (MNAR data)
- **Fix:** Investigate the pattern of missingness first

5. Dropping rows with `dropna()` with no subset

- **Problem:** Can silently wipe out most of the dataset
- **Fix:** Always specify `subset=` for target columns

6. Using `mean()` to fill skewed data

- **Problem:** Mean is pulled by outliers (e.g., salary, price)
- **Fix:** Use `median()` for skewed distributions

7. Ignoring leading/trailing spaces in text

- **Problem:** "USA" and "USA " are treated as different categories
- **Fix:** Always `.str.strip()` text columns early

8. Comparing dates/strings without converting types

- **Problem:** `"2024-01-01" > "2023-12-31"` works by luck as text, but breaks for other formats
- **Fix:** Always convert with `pd.to_datetime()` first

9. Not resetting the index after filtering

- **Problem:** Leftover gaps in the index confuse `iloc[]` and looping
- **Fix:** Use `df.reset_index(drop=True)` after major filtering

10. Deleting outliers without investigation

- **Problem:** May remove real, valuable rare events
- **Fix:** Investigate whether it's an error or a fact first

11. Applying one-hot encoding to high-cardinality columns

- **Problem:** A "city" column with 5,000 unique values creates 5,000 new columns
- **Fix:** Group rare categories first, or use label/target encoding

12. Not saving intermediate cleaned versions

- **Problem:** Losing hours of cleaning work if the notebook kernel crashes
- **Fix:** Periodically `df.to_csv()` checkpoints during a long cleaning session

Chapter 17: Data Cleaning Interview Questions

17.1 Beginner Level

Q1. What is data cleaning and why is it important?

It is the process of fixing/removing incorrect, incomplete, or inconsistent data. It matters because "garbage in, garbage out" — poor data quality directly produces poor analysis and models.

Q2. How do you check for missing values in a DataFrame?

`df.isnull().sum()` gives a per-column count; `df.isnull().mean()*100` gives the percentage.

Q3. What is the difference between `dropna()` and `fillna()`?

`dropna()` removes rows/columns with missing values; `fillna()` fills them in with a chosen value/strategy.

Q4. How do you find and remove duplicate rows?

`df.duplicated()` to detect, `df.drop_duplicates()` to remove; use `subset=` to key on specific columns.

Q5. What does `df.info()` show you?

Row count, column names, non-null counts per column, and data types — a quick health check of the dataset.

17.2 Intermediate Level

Q1. Explain MCAR, MAR, and MNAR.

MCAR: missingness has no relation to any data (pure randomness). MAR: missingness relates to another observed column (e.g., younger customers skip income). MNAR: missingness relates to the missing value itself (e.g., high earners refuse to disclose income).

Q2. When would you use median instead of mean to fill missing values?

When the column is skewed or has outliers — mean gets pulled toward extreme values, while median stays representative of the "typical" value.

Q3. How do you detect outliers, and what do you do with them?

Use IQR ($Q1 - 1.5 * IQR$ to $Q3 + 1.5 * IQR$) or Z-score ($|z| > 3$) methods to detect them, then investigate whether each is a genuine value or a data error before deciding to remove, cap, transform, or keep it.

Q4. What is the difference between `.loc[]` and `.iloc[]`?

`.loc[]` selects by label (column/index names); `.iloc[]` selects by integer position, like array indexing.

Q5. Why is chained indexing (`df[df.x>5]["y"]=1`) discouraged?

It can create an ambiguous copy-vs-view situation in Pandas, triggering `SettingWithCopyWarning` and sometimes failing to actually update the original DataFrame. Use `.loc[]` instead.

Q6. How would you standardize inconsistent category values like "NY", "New York", "new york"?

Strip whitespace, normalize case, then map all known variants to one canonical value using `.replace()` with a mapping dictionary.

17.3 Advanced Level

Q1. How would you design a cleaning pipeline that is repeatable across many similar datasets?

Wrap each cleaning step (rename columns, fix types, handle nulls, drop duplicates, validate ranges) into small, testable functions or a scikit-learn Pipeline/ColumnTransformer, so the same logic can run consistently on new incoming data.

Q2. How do you decide between deleting, imputing, or flagging missing data at scale?

Base the decision on: percentage missing, the missingness mechanism (MCAR/MAR/MNAR), the importance of the column to the target task, and how much data can be spared — document the decision so it is auditable later.

Q3. How would you validate that a cleaning pipeline hasn't introduced new bugs?

Compare row counts and key summary statistics before/after each step, write unit tests/assertions (e.g., "no nulls remain in required columns", "no negative prices"), and keep a data-quality report generated automatically after each pipeline run.

Q4. A dataset has millions of rows and `.apply()` is too slow — what do you do?

Prefer vectorized Pandas/NumPy operations first; if still slow, use tools like Numba, Dask, or vectorized string operations, or push heavy computation into SQL/Spark before it reaches Pandas.

Q5. How do you handle schema drift when the same data source changes column names or types over time?

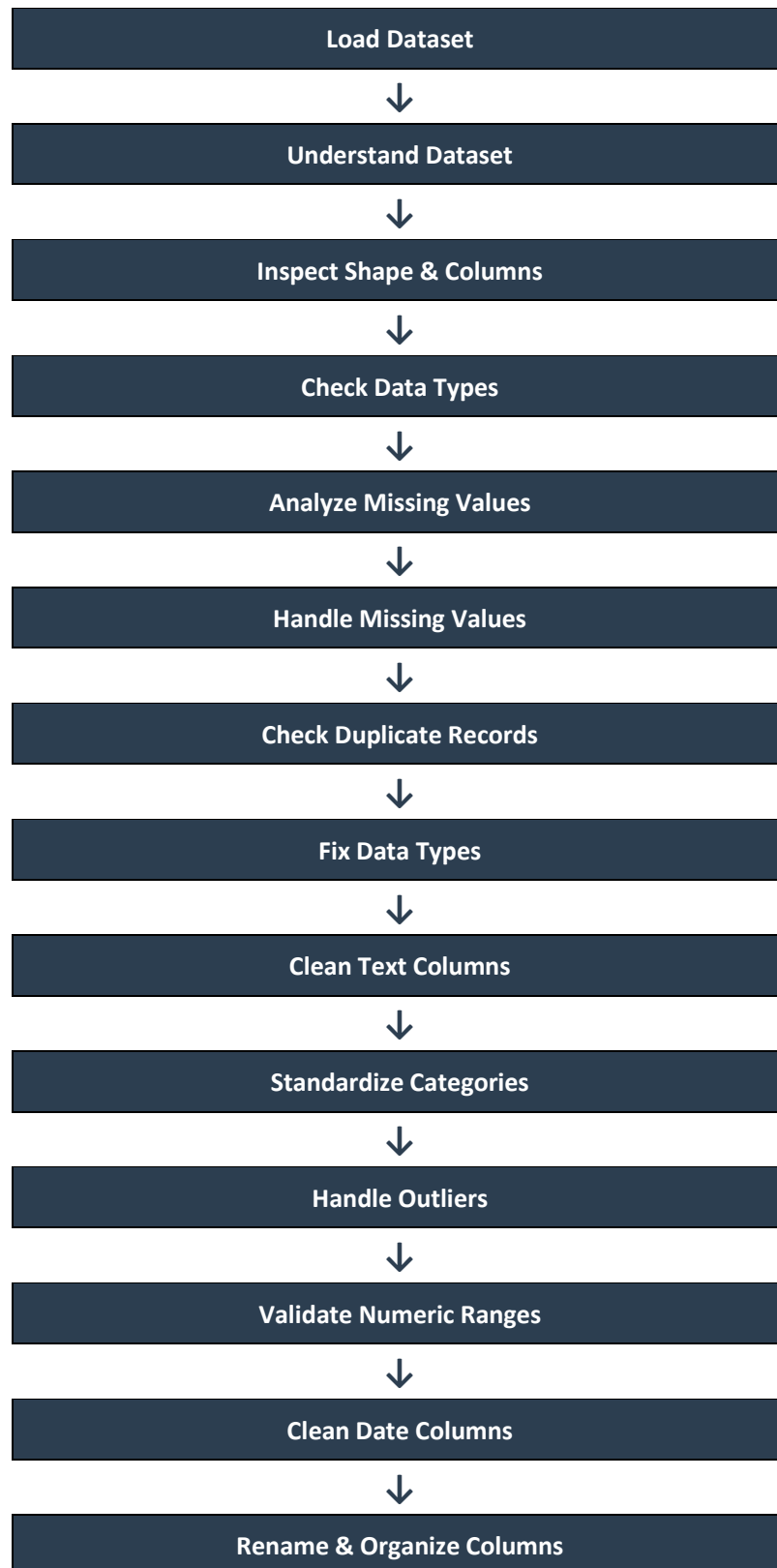
Build a schema-validation step (e.g., using pandera or pydantic) at the start of the pipeline that checks expected columns/types exist, and fails loudly (or logs a warning) rather than silently processing wrong data.

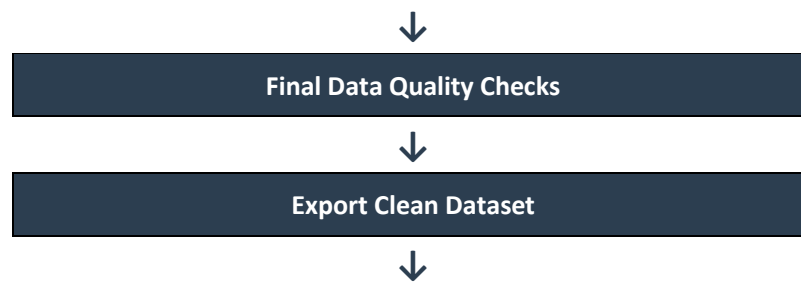
INTERVIEW TIP

In interviews, always structure answers as: Definition -> Why it matters -> A concrete example from a dataset (housing/banking/employee etc.) -> How you'd verify the fix worked. This shows applied thinking, not just memorized syntax.

Chapter 18: Complete Data Cleaning Roadmap for Any Dataset

This is the master checklist. Print it, pin it, or bookmark it — follow these steps in order on any new dataset you ever receive, whether it's housing, banking, healthcare, or retail.





18.1 Detailed Notes for Every Step

Load Dataset

- **Common functions:** `pd.read_csv/read_excel/read_sql`
- **Why it's done:** Establishes the raw starting point
- **Common mistake:** Wrong encoding/dtype guesses at load time cause downstream bugs
- **Best practice:** Use `na_values`, `dtype`, `parse_dates` arguments proactively
- **Can be skipped when:** Never — always required

Understand Dataset

- **Common functions:** `head()`, `tail()`, `sample()`
- **Why it's done:** Builds intuition about what "normal" looks like
- **Common mistake:** Only checking `head()` misses messy rows further down
- **Best practice:** Always check `sample(10)` too
- **Can be skipped when:** Never — always required

Inspect Shape & Columns

- **Common functions:** `shape`, `columns`
- **Why it's done:** Confirms you loaded what you expected
- **Common mistake:** Assuming column order/names without checking
- **Best practice:** Compare shape against source documentation/row count
- **Can be skipped when:** Skip only if automated pipeline already validates schema

Check Data Types

- **Common functions:** `dtypes`, `info()`
- **Why it's done:** Reveals numbers-as-text, wrong date types
- **Common mistake:** Trusting Pandas' auto-inferred types blindly
- **Best practice:** Cross-check a few columns manually
- **Can be skipped when:** Never — always required

Analyze Missing Values

- **Common functions:** `isnull().sum()`, `isnull().mean()`
- **Why it's done:** Quantifies the scale of the missing-data problem
- **Common mistake:** Not checking for disguised missing values ("`N/A`", "`?`")

- **Best practice:** Check `.unique()` on suspicious columns too
- **Can be skipped when:** Never — always required

Handle Missing Values

- **Common functions:** `dropna()`, `fillna()`, `groupby().transform()`
- **Why it's done:** Produces a usable, complete dataset
- **Common mistake:** Filling before understanding WHY data is missing
- **Best practice:** Use decision framework from Chapter 4
- **Can be skipped when:** Skip if the column will be dropped anyway

Check Duplicate Records

- **Common functions:** `duplicated()`, `drop_duplicates()`
- **Why it's done:** Prevents double-counting in aggregations
- **Common mistake:** Checking only exact duplicates, missing key-based dupes
- **Best practice:** Always check `subset=` on business key columns
- **Can be skipped when:** Skip if data source guarantees uniqueness (e.g., DB primary key)

Fix Data Types

- **Common functions:** `astype()`, `to_numeric()`, `to_datetime()`
- **Why it's done:** Enables correct math, sorting, filtering
- **Common mistake:** `astype(int)` crashing on remaining NaNs
- **Best practice:** Convert AFTER missing values are handled where possible
- **Can be skipped when:** Skip for columns already correctly typed

Clean Text Columns

- **Common functions:** `.str.strip()`/`lower`/`replace`
- **Why it's done:** Prevents "same value" being treated as different categories
- **Common mistake:** Forgetting hidden whitespace or non-ASCII characters
- **Best practice:** Always `.str.strip()` as a default first step
- **Can be skipped when:** Skip for purely numeric datasets

Standardize Categories

- **Common functions:** `replace()`, `map()`, `value_counts()`
- **Why it's done:** Groups synonyms into one consistent label
- **Common mistake:** `.map()` silently NaN-ing unmapped values
- **Best practice:** Check `unique()` before mapping
- **Can be skipped when:** Skip if categories are already an enforced enum/dropdown at entry

Handle Outliers

- **Common functions:** IQR, Z-score, `clip()`, domain rules
- **Why it's done:** Prevents extreme values from distorting statistics/models
- **Common mistake:** Deleting outliers without checking if they're genuine

- **Best practice:** Investigate before removing — see Chapter 8
- **Can be skipped when:** Skip for columns where extreme values are expected/valid

Validate Numeric Ranges

- **Common functions:** Domain-based min/max checks
- **Why it's done:** Catches logically impossible values (negative age)
- **Common mistake:** Assuming `describe()` alone reveals all range errors
- **Best practice:** Always apply explicit domain business rules
- **Can be skipped when:** Skip if no domain constraints exist

Clean Date Columns

- **Common functions:** `to_datetime()`, `dt` accessor
- **Why it's done:** Enables correct time-based analysis
- **Common mistake:** Mixed date formats parsed incorrectly
- **Best practice:** Use `errors="coerce"` and inspect `NaT` rows
- **Can be skipped when:** Skip if no date columns exist

Rename & Organize Columns

- **Common functions:** `rename()`, reordering
- **Why it's done:** Improves readability & downstream code stability
- **Common mistake:** Inconsistent naming (spaces, mixed case) across projects
- **Best practice:** Adopt `snake_case` as a team standard
- **Can be skipped when:** Skip if already following a naming standard

Final Data Quality Checks

- **Common functions:** `isnull().sum()`, `duplicated().sum()`, `describe()`
- **Why it's done:** Confirms the cleaning actually worked
- **Common mistake:** Skipping this and assuming all steps worked
- **Best practice:** Always re-run the Chapter 3 "starter kit" one final time
- **Can be skipped when:** Never — always required

Export Clean Dataset

- **Common functions:** `to_csv(index=False)`, `to_excel()`
- **Why it's done:** Delivers a reusable, trustworthy dataset
- **Common mistake:** Forgetting `index=False`, adding junk columns
- **Best practice:** Version your output file name (e.g., `_clean_v1`)
- **Can be skipped when:** Never — always required

Chapter 19: Practice Datasets and Exercises

Apply the roadmap from Chapter 18 to each of these scenarios. Try to solve them yourself before checking the techniques used in earlier chapters.

Housing Prices Dataset

Columns: price, area_sqft, bedrooms, city, listed_date, garage_size

- Some prices are stored as text with "\$" and commas — convert to numeric.
- garage_size is missing for homes with no garage — should this be 0 or NaN?
- Detect and handle outlier prices (mansions vs data errors) using IQR.
- Standardize city names ("NYC", "New York", "new york").

Customer Information Dataset

Columns: name, email, phone, city, signup_date, membership_tier

- Split full name into first_name and last_name.
- Validate email format using a regex and flag invalid ones.
- Standardize phone numbers into a consistent 10-digit format.
- Fill missing membership_tier with the most frequent tier per city.

Banking Transactions Dataset

Columns: account_id, transaction_amount, transaction_date, account_type, is_fraud

- Find transactions with amount ≤ 0 (likely errors) and investigate.
- Standardize account_type categories (case & whitespace issues).
- Convert is_fraud from "Y"/"N" text into a real boolean.
- Validate that transaction_date is never before account_open_date.

Employee Records Dataset

Columns: emp_id, name, department, salary, join_date, rating

- Impute missing salary using the department median (group-wise imputation).
- Detect and cap extreme salary outliers using the IQR method.
- Convert join_date to datetime and compute tenure_days.
- Bin employees into age_group categories using pd.cut().

Medical Records Dataset

Columns: patient_id, age, gender, bmi, blood_pressure, diagnosis_date

- Validate that age falls within a realistic range (0-120).
- Fill missing BMI using group-wise median by age_group.
- Standardize gender labels ("M", "male", "MALE") into one format.
- Flag any diagnosis_date values that fall in the future.

Product Inventory Dataset

Columns: sku, product_name, category, price, quantity, supplier

- Clean and standardize SKU codes (remove dashes/spaces, uppercase).
- Convert price from text ("-\$1,299.00") to a proper float.
- Group rare categories (fewer than 10 products) into "Other".
- Flag rows with negative quantity as data entry errors.

Restaurant Reviews Dataset

Columns: review_id, restaurant_name, city, rating, comment, review_date

- Remove non-alphanumeric noise/emojis from the comment column.
- Investigate ratings outside the expected 1-5 scale.
- Detect near-duplicate reviews (same reviewer, similar comment) using fuzzy matching.
- Extract review_year and review_month from review_date for trend analysis.

PRO TIP

For real practice, download similar free datasets from Kaggle, UCI Machine Learning Repository, or data.gov, and intentionally introduce errors (duplicate rows, blank cells, typos) to build your own "messy" practice files.

Chapter 20: Final 10-15 Minute Revision Sheet

Read this page right before an interview or exam. It compresses the entire guide into the essentials.

The Roadmap (Memorize This Order)

Load → Understand → Shape/Columns → Data Types → Analyze Missing → Handle Missing → Duplicates → Fix Types → Clean Text → Standardize Categories → Outliers → Validate Ranges → Clean Dates → Rename/Organize → Final QC → Export

Missing Values — 3 Strategies

- Delete (dropna) — when missing % is small and random (MCAR)
- Impute (fillna, mean/median/mode, group-wise, interpolation) — most common approach
- Flag (add an is_missing column) — when missingness itself carries information

Outliers — 4 Options

- Remove — if it's a genuine data error
- Cap/Winsorize (clip()) — keep the row but limit its influence
- Transform (log) — reduce the skew impact
- Keep — if it's a valid, meaningful extreme value

Key One-Liners to Remember

Concept	One-Liner
Missing check	<code>df.isnull().sum()</code>
Duplicate check	<code>df.duplicated().sum()</code>
Drop duplicates by key	<code>df.drop_duplicates(subset=["id"], keep="last")</code>
Safe numeric conversion	<code>pd.to_numeric(df["col"], errors="coerce")</code>
Safe date conversion	<code>pd.to_datetime(df["col"], errors="coerce")</code>
Group-wise fill	<code>df.groupby("grp")["col"].transform(lambda x: x.fillna(x.median()))</code>
IQR outlier bounds	<code>Q1-1.5*IQR to Q3+1.5*IQR</code>
Safe filtered assignment	<code>df.loc[condition, "col"] = value</code>
Standardize text	<code>.str.strip().str.lower()</code>
Export without junk column	<code>df.to_csv("file.csv", index=False)</code>

Interview Golden Rules

1. Always explain WHY before HOW — never jump straight to code.

2. Mention MCAR/MAR/MNAR when discussing missing data — it signals depth.
3. Never say "I always just drop nulls" — show you understand trade-offs.
4. Always mention validating your fix afterward (re-check isnull/duplicated/describe).
5. Give a real dataset example (housing, banking, employees) to ground your answer.

INTERVIEW TIP

If you remember nothing else: Detect the problem -> Understand WHY it exists -> Choose a method with a clear reason -> Validate the fix worked. This 4-step mental model answers almost any data cleaning interview question.

You're Ready

You now have a complete, structured framework for cleaning any real-world dataset — from missing values and duplicates to outliers, categories, and dates. Revisit the End-to-End Example in Chapter 14 whenever you need a refresher on how all these pieces fit together in a single, real cleaning script. Good luck!